

exp-mlkem-sepolyvec8-ntt-k4

研究型 AscendC 算子实现方案

ascendc 工程（预研）

2026-06-10

摘要

本文档归纳 `examples/incubating/exp-mlkem-sepolyvec8-ntt-k4/` 的实现方案（规格书，非源码全文）。算子语义为 ML-KEM 语境下 $k=8$ 批 NTT：一次 MIX kernel 对 8 条互不相同、随机生成的 Z_q 多项式完成 $\text{NTT}(\cdot) \bmod 3329$ 。工程壳对齐 `thirdparty/merged_kyber` 手写 FSM 与已调通探针 `ascendc-tests/fix-merged-kyber-ntt256-limb6-poly8-s123/`（6bit limb、单 TPipe batch Split/Merge、 $2\times\text{AicMmad}$ 、Barrett 归约），不再以 Gitee `MatmulLeakyReluInvocation` 的 `Matmul<>` 融合模板或 `exp-mlkem-f203-stage12-encode-matmul-mix` 为 MIX 依据（后者已废弃）。

目录

1	定位与边界	2
1.1	用例性质	2
1.2	目标平台	2
2	参考实现与已废弃路径	2
2.1	主参考：poly8-s123 探针（实现照抄此模式）	2
2.2	语义对照：sepolyvec8_ntt_f203（仅形状与 NTT 定义）	3
2.3	已废弃（勿再 fork）	3
3	计算语义：merged_kyber 6bit 三段式（$k=8$）	3
3.1	GM 工作区布局（与 poly8-s123 tiling.h 一致）	3
3.2	MIX 同步（FSM）	3
4	I/O 与 Golden 契约	4
4.1	输入：8 条互不相同的随机多项式（强制）	4
4.2	文件名（gen_data 与 main 逐字一致）	4
4.3	Golden 生成	4
5	AI Core 资源分工	4

目录	2
6 Tiling 与核配置	4
6.1 与 Matmul 融合模板无关的 launch 语义	4
6.2 批维常量	5
6.3 单 TPipe 硬约束（摘自调通纪要）	5
7 各阶段实现要点	5
7.1 Stage1: batch Split	5
7.2 Stage2: 双次 AicMmad	5
7.3 Stage3: batch Merge + Barrett	5
8 Python 对照组与 run.sh	5
8.1 标准三步（对齐 KernelLaunch 样例，非 ntt_study 预置 bin）	5
8.2 gen_data.py 要求	6
9 计划目录与迁移路径	6
10 分步验收	6
11 风险与依赖	6

1 定位与边界

1.1 用例性质

- **研究/预研**: 代码落在 `examples/incubating/exp-*`; Golden 仅验 I/O (`max_abs_diff=0`)。
- **算法片段**: 对 8 条多项式分别做 Kyber NTT ($n=256, q=3329$); 与 FIPS 203 KeyGen 中 $\hat{s} \leftarrow \text{NTT}(s), \hat{e} \leftarrow \text{NTT}(e)$ 的批维规模一致 ($k=4$ 时共 8 路), 但首版输入不要求固定 $s||e$ 拼接——见 §5。
- **不在本用例**: 完整 KeyGen、内积、SHA3/采样、Encaps/Decaps、FIPS 204、多 NPU 性能调优。

1.2 目标平台

- **CANN / AscendC**: 社区版 9.0.0; 范式 **KernelLaunch** (SIMD)。
- **硬件**: Atlas A2; 不支持 310P 等其它型号。
- **核规模**: 1 个 AI Core (`blockDim=1`); 其内 1 Cube + 2 Vector (1:2)。
- **内核类型**: `KERNEL_TYPE_MIX_AIC_1_2`; 单次 **launch** 跑完全链路 (禁止 CPU 两趟 launch 绕过同步, 见已废弃 `stage12`)。
- **验证**: `WSL run.sh cpu/sim`; 有 A2 时补 `np`。

2 参考实现与已废弃路径

2.1 主参考: poly8-s123 探针 (实现照抄此模式)

路径: `ascendc-tests/fix-merged-kyber-ntt256-limb6-poly8-s123/`。

- `kPolys=8, mRows=16`; `AivSplit / AivMerge` 各一个 **TPipe**, `tileLength = kPolys × (n/2)`。
- Stage2: `AicMmad(16, 256, 256)`, `Process(ws+A0, S0, M0)`、`Process(ws+A1, S0, M1)`; 中间紧凑 `[16, 256] int32`, 非 `[16, 512]`。
- Stage3: batch Merge + Barrett (`barrett_mul_vec_runtime`), 从 A0/A1 行 $2p / 2p+1$ 取数, 槽 2/3 置零 (等价单 poly limb6)。
- CPU 对拍: 8 行 NTT 各自与单 poly merged-kyber-ntt256-limb6 golden 一致 (探针内 8 行系数相同; 本 **exp** 改为 8 行互异随机, 见 §5)。
- 经验纪要: `docs/research/20260610-merged-kyber-6bit-双 poly 批量化 NTT 调通纪要.md`。

2.2 语义对照: sepolyvec8_ntt_f203 (仅形状与 NTT 定义)

路径: `thirdparty/ntt_study/examples/mlkem/deliverables/sepolyvec8_ntt_f203/`。

- 批维 $K = 8$; 环 $n = 256$, $q = 3329$ 。
- ONNX 交付路径为 `fp16 MatMul + RouteA Gather`; 本 **exp** 不实现该路径, 仅以「8 路批 NTT 输出 `[8,256] int32`」为语义对齐点。
- Golden: 每行 $\text{MlKemNtt}(a_i) \bmod q$ (Python 参考实现交叉校验)。

2.3 已废弃 (勿再 fork)

- `exp-mlkem-f203-stage12-encode-matmul-mix`: LeakyRelu 融合 MIX、CPU 两趟 launch、sim encode 写 GM 失败。
- 本目录旧稿中的 `Matmul<int8,int8,int32> + REGIST_MATMUL_OBJ + Stage3 RouteA` (从 `[16,512] Gather`): 与 `limb6 / poly8` 已证伪的中间布局不一致。

3 计算语义: merged_kyber 6bit 三段式 ($k=8$)

对每条多项式 $a_i \in Z_q^n$, 目标 $y_i = \text{NTT}(a_i) \bmod q$ 。拓扑对齐 `merged-kyber-ntt256-limb6` 的 D' 全链路, 批维扩展为 8:

1. **Stage1 Split**(AIV): `src [8,256] int32` $\rightarrow$ `S0 [16,256] int8`, 行序 `[hi0,lo0,...,hi7,lo7]`; 6bit limb: $hi = \lfloor a/64 \rfloor$, $lo = a - 64 \cdot hi$ (掩码 63)。
2. **Stage2 Mmad** (AIC): $A_0 = S_0 \cdot M_0$, $A_1 = S_0 \cdot M_1$; M_0, M_1 为 LUT `M4.bin` 中前两块 256×256 `int8`; 输出 `A0,A1` 各 `[16,256] int32` (紧凑, 无 512 列展开)。
3. **Stage3 Merge + Barrett** (AIV): 按 `poly` 从 `A0/A1` 取行 $2p$ 、 $2p+1$ 半行, 向量移位相加后经 Barrett 归约到 $[0, q)$, 写出 `dst [8,256] int32`。

3.1 GM 工作区布局 (与 `poly8-s123 tiling.h` 一致)

`ws`: M_0, M_1, M_2, M_3 (各 256^2 `int8`), $S_0[16, 256], \dots, A_0, A_1[16, 256]$ `int32`。

LUT 文件对外仍可为 `mat_b_lut_gm.bin` 形状 `[256,512] int8` (四块 256×256 拼接), 与 `limb6 / ntt_study` 常量布局兼容。

3.2 MIX 同步 (FSM)

- `CrossCoreSetFlag / CrossCoreWaitFlag`: Stage1 完成 $\rightarrow$ Stage2 $\rightarrow$ Stage3 (标志位与 `merged_kyber / poly8` 探针一致, 如 `kSyncStage1Done=7`、`kSyncStage2Done=9`)。
- AIV 两子核: Split 与 Merge 各跑 `subCoreIdx` $\in \{0,1\}$ 半行; 禁止在 `for (p)` 内构造新 `AivSplit/AivMerge` (`AllocEventID` 耗尽 / 挂死)。

4 I/O 与 Golden 契约

4.1 输入：8 条互不相同的随机多项式（强制）

- `gen_data.py` 使用固定 `seed`（文档写明，如 `seed=20260610`），生成 8 个独立系数向量，每行 $a_{i,j} \in [0, q)$ 或 `int32` 表示的 Z_q 系数，禁止 `np.tile` 同一 `poly` 8 次。
- 可选开关 `--use-ntt-fixed-se`：复现 `ntt_study` 确定性 $s||e$ （回归用，非默认）。

4.2 文件名（`gen_data` 与 `main` 逐字一致）

路径	形状	dtype	语义
<code>input/se_polyvec_gm.bin</code>	<code>[8, 256]</code>	<code>int32</code>	8 条互异随机 <code>poly</code>
<code>input/mat_b_lut_gm.bin</code>	<code>[256, 512]</code>	<code>int8</code>	$M_0 M_1 M_2 M_3$ （ <code>limb6</code> 布局）
<code>output/output.bin</code>	<code>[8, 256]</code>	<code>int32</code>	内核 NTT 结果
<code>output/golden.bin</code>	<code>[8, 256]</code>	<code>int32</code>	Python 逐行 NTT 基准

禁止默认使用 `ntt_study` 的 `input0.bin/input1.bin` 作 `launch` 路径（仅 `--cross-check-ntt-study` 可选读入对照）。

4.3 Golden 生成

1. 主路径：Python `ntt_forward_kyber` / `f203_limb6_batch_ref`（与 `poly8` 探针三段式 `int8` 路径一致）写出 `golden.bin`。
2. 交叉校验：对每行再算 `MlkemNtt`（`ntt_study` 公式）；两者 `max_abs_diff=0` 后再落盘。
3. 验收：`verify_result.py` 要求 严格相等（`int32`, `max_abs_diff=0`）。

5 AI Core 资源分工

资源	落点	本用例
Scalar	<code>*_tiling.cpp</code> 、 <code>mmad_custom.cpp</code> 入口	<code>tiling</code> 、 <code>workspace</code> 偏移、FSM
Cube (AIC)	<code>AicMmad</code> （ <code>merged_kyber</code> ）	$16 \times 256 \times 256$, <code>int8</code> → <code>int32</code> , 两次（ M_0, M_1 ）
Vector (AIV)	<code>AivSplit</code> 、 <code>AivMerge</code> 、 <code>ntt_vec.hpp</code>	6bit split; Merge+Barrett
同步	<code>CrossCore*</code>	单趟 <code>launch</code> 三段流水

6 Tiling 与核配置

6.1 与 `Matmul` 融合模板无关的 `launch` 语义

- `blockDim = 1`: 1 个 AI Core。

- 无 SetDim(2) 的 Matmul 融合 usedCoreNum 语义; Vector 双核由 MIX 类型 MIX_AIC_1_2 与算子内 subCoreIdx 分工。

6.2 批维常量

- $kPolys = 8, n = 256, mRows = 2 \times kPolys = 16$ 。
- Stage1 tileLength = $kPolys \times (n/2) = 1024$ (每 AIV 半行一次 split_vec)。
- Stage2 Cube: 左矩阵 $M=16, K=N=256$; 遵守 int8 入 int32 出最小 tile (实现时对照 CANN 文档, poly8 探针已验证 aicore=1)。

6.3 单 TPipe 硬约束 (摘自调通纪要)

1. 每个 AivSplit/AivMerge 实例: Init() 内 仅一次 pipe.InitBuffer。
2. for (p) 只允许出现在 CopyIn/CopyOut 的 GM gather/scatter, 不允许循环内 new 算子类。
3. Merge CopyIn: 按 poly 读 A0/A1 行 $2p, 2p+1$; 缓冲区先 Duplicate(0), 槽 2/3 保持零。

7 各阶段实现要点

7.1 Stage1: batch Split

参照 fix-...-poly8-s123/aiv_func.hpp 中 AivSplit: 一趟 split_vec 覆盖 8 poly 的半行系数; GM 写 S0 行 $2p, 2p+1$ 。

7.2 Stage2: 双次 AicMmad

参照 mmad_custom.cpp: AicMmad 读 S0 与 M0/M1, 写 ws+A0、ws+A1; 不写 [16,512] 宽矩阵。

7.3 Stage3: batch Merge + Barrett

参照 AivMerge::Compute: 每 poly 独立 Merge 拓扑 + 两次 barrett_mul_vec_runtime; tmp_poly 缓冲避免 GM 写 rvalue 问题。

8 Python 对照组与 run.sh

8.1 标准三步 (对齐 KernelLaunch 样例, 非 ntt_study 预置 bin)

```
rm -rf input output && mkdir -p input output
python3 scripts/gen_data.py          # 8 随机互异 poly + golden
./ascendc_kernels_bbit               # cpu / sim / npu
python3 scripts/verify_result.py output/output.bin output/golden.bin
```

SIM 模式须 source scripts/camodel_sim_log.sh, 避免根目录 core*.dump 污染 (poly8 探针惯例)。

8.2 gen_data.py 要求

1. `rng = np.random.default_rng(seed)`; 生成 `[8, 256]`, 行之间不得相等 (断言 `len(unique(rows))==8`)。
2. 写 `mat_b_lut_gm.bin`: 与 `merged-kyber-ntt256-limb6 / kMlKemLimb6Ntt_T_i8` 四块 `int8` 一致。
3. Golden: 逐行 NTT + limb6 三段参考双路径校验。

9 计划目录与迁移路径

`exp-mlkem-sepolyvec8-ntt-k4/`

```
exp-mlkem-sepolyvec8-ntt-k4-实现方案.tex / .pdf
sepolyvec8_ntt_custom_tiling.cpp    # 待按 poly8 重写
sepolyvec8_ntt_custom.cpp          # 待: merged_kyber FSM + Aiv*/Aic*
main.cpp, data_utils.h, CMakeLists.txt, run.sh
scripts/gen_data.py                # 待: 8 互异随机 poly
scripts/verify_result.py
```

推荐实施顺序: 从 `ascendc-tests/fix-merged-kyber-ntt256-limb6-poly8-s123/` 复制骨架
→ 改 `gen_data.py` 为 8 互异随机 → 迁入本 `exp` 目录 → CPU 对拍 → sim。

10 分步验收

步	内容	验收
S0	本方案 PDF + 废弃旧 Matmul 路线说明	文档就绪
S1	<code>gen_data</code> : 8 互异随机 + golden	逐行 MlKemNtt 一致
S2	迁入 poly8 AicMmad + workspace	中间 A0/A1 对拍
S3	batch AivSplit	S0 对拍
S4	batch AivMerge + Barrett	output vs golden, 0 差
S5	MIX 单趟 launch (cpu/sim)	无挂死、无 event 耗尽
S6	A2 npu	后续

11 风险与依赖

- **旧代码债务:** 当前 `sepolyvec8_ntt_custom.cpp` 仍为 Matmul 融合草稿, 须整文件替换, 不可局部修补。
- **int8 LUT 饱和:** 以 limb6 四块 `int8` 为准, 与 fp16 ONNX 路径可能有差, 以 Python limb6 参考为准。
- **UB:** `kPolys=8` 时 `tileLength=1024`, 须确认 Vector 缓冲与 poly8 探针一致。

- 关联文档:docs/research/20260610-f203-ntt-mix-merged-kyber 开发策略.md、qa/2026-06/2026-06-路线与 limb6.md。